

AI ASSISTED CODING

LAB TEST – 03

NAME : J.KEERTHI PRIYA

ROLL NO : 2403A510G4

BATCH : 06

Q1:

PROMPT :

A retail company wants to recommend similar products to customers based on what they are currently viewing.

Design an algorithm (with AI assistance) that can automatically recommend similar items based on product descriptions.

CODE :

```

task1.py > ...
1  # AI-assisted Product Recommendation System (Retail Sector)
2
3  from sklearn.feature_extraction.text import TfidfVectorizer
4  from sklearn.metrics.pairwise import cosine_similarity
5  import pandas as pd
6
7  # -----
8  # Sample Product Dataset
9  # -----
10 data = {
11     'Product_ID': [1, 2, 3, 4, 5],
12     'Product_Name': [
13         'Red Cotton T-Shirt',
14         'Blue Denim Jeans',
15         'Cotton Casual Shirt',
16         'Leather Wallet',
17         'Formal Black Shoes'
18     ],
19     'Description': [
20         'Comfortable red cotton t-shirt for daily wear',
21         'Stylish blue denim jeans for men',
22         'Soft cotton shirt perfect for casual outings',
23         'Premium leather wallet with multiple card slots',
24         'Elegant black shoes for formal occasions'
25     ]
26 }
27
28 # Create DataFrame
29 df = pd.DataFrame(data)
30
31 # -----
32 # Step 1: Convert text into TF-IDF vectors
33 # -----
34 vectorizer = TfidfVectorizer(stop_words='english')
35 tfidf_matrix = vectorizer.fit_transform(df['Description'])
36
37 # -----

```

```

task1.py > ...
29 df = pd.DataFrame(data)
30
31 # -----
32 # Step 1: Convert text into TF-IDF vectors
33 # -----
34 vectorizer = TfidfVectorizer(stop_words='english')
35 tfidf_matrix = vectorizer.fit_transform(df['Description'])
36
37 # -----
38 # Step 2: Compute cosine similarity
39 # -----
40 cosine_sim = cosine_similarity(tfidf_matrix, tfidf_matrix)
41
42 # -----
43 # Step 3: Recommend similar products
44 # -----
45 def recommend(product_name):
46     idx = df[df['Product_Name'] == product_name].index[0]
47     sim_scores = list(enumerate(cosine_sim[idx]))
48     sim_scores = sorted(sim_scores, key=lambda x: x[1], reverse=True)
49     sim_scores = sim_scores[1:3] # Top 2 similar products
50     product_indices = [i[0] for i in sim_scores]
51     return df[['Product_ID', 'Product_Name']].iloc[product_indices]
52
53 # -----
54 # Step 4: Test the recommendation system
55 # -----
56 print("Product Recommendations for 'Red Cotton T-Shirt':")
57 print(recommend('Red Cotton T-Shirt'))

```

OUTPUT :

```

Lotion - Predicted Next Stock: 72.00
PS C:\Users\keerthi priya\Desktop\ai lab test - 3> & "C:/Users/keerthi priya/AppData/Local/Microsoft/WindowsApps/python3.11.exe" "C:/Users/keerthi priya/Desktop/ai lab test - 3/task1.py"
Product Recommendations for 'Red Cotton T-Shirt':
  Product_ID  Product_Name
2          3  Cotton Casual Shirt
1          2   Blue Denim Jeans
PS C:\Users\keerthi priya\Desktop\ai lab test - 3> 

```

OBSERVATION :

1. AI model converts text into numerical vectors (embeddings).
2. Cosine similarity measures closeness between products.
3. Similar items (like “headphones”) score higher for earphones query.
4. AI assistance improves accuracy without manual rule creation.
5. This algorithm can scale for large product catalogs easily.

Q2.

PROMPT :

An education company wants to identify top-performing students efficiently from large data using **data structures**.

AI is used to analyze patterns and predict performance trends

Generate a Python program that uses data structures (like heap and dictionary) to analyze student scores and predict future marks using AI regression model.

CODE :

```
task2.py > ...
1  # AI-Assisted Student Performance Analysis using Data Structures
2
3  import heapq
4  from sklearn.linear_model import LinearRegression
5  import numpy as np
6
7  # Step 1: Store student data using dictionary
8  students = {
9      "Amit": [80, 85, 90],
10     "Priya": [92, 89, 95],
11     "Ravi": [78, 74, 70],
12     "Keerthi": [88, 91, 84],
13     "Rahul": [65, 70, 68]
14 }
15
16 # Step 2: Compute average marks and store in a list
17 averages = []
18 for name, marks in students.items():
19     avg = sum(marks) / len(marks)
20     averages.append((avg, name))
21
22 # Step 3: Use Heap (data structure) to find top 3 students
23 top_students = heapq.nlargest(3, averages)
24
25 print("Top 3 Students:")
26 for avg, name in top_students:
27     print(f"{name} - Average Marks: {avg}")
28
29 # Step 4: AI Model (Linear Regression) to predict next exam marks
30 print("\nAI Predicted Performance:")
31 for name, marks in students.items():
32     X = np.arange(len(marks)).reshape(-1, 1)
33     y = np.array(marks)
34     model = LinearRegression()
35     model.fit(X, y)
36     next_score = model.predict([[len(marks)]])[0]
37     print(f"{name} - Predicted Next Score: {next_score:.2f}")
```

OUTPUT:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\keerthi priya\Desktop\ai lab test - 3> & "C:/Users/keerthi priya/AppData/Local/Microsoft/WindowsApps/python3.11.exe" "c:/Users/keerthi priya/Desktop/ai lab test - 3/task2.py"
Top 3 Students:
Priya - Average Marks: 92.0
Keerthi - Average Marks: 87.66666666666667
Amit - Average Marks: 85.0

AI Predicted Performance:
Amit - Predicted Next Score: 95.00
Priya - Predicted Next Score: 95.00
Ravi - Predicted Next Score: 66.00
Keerthi - Predicted Next Score: 83.67
Rahul - Predicted Next Score: 70.67
PS C:\Users\keerthi priya\Desktop\ai lab test - 3>
```

OBSERVATION :

1. Student records are stored efficiently using a dictionary data structure.
2. Heap data structure is used to quickly retrieve top-performing students.
3. AI regression model predicts student's next score automatically.
4. Data structures improve efficiency in handling multiple student records.
5. AI helps discover patterns and future performance trends.
6. The combination of heap and regression gives both ranking and prediction.
7. The algorithm can be scaled for thousands of students easily.
8. Useful for institutions to identify high achievers and students needing help.
9. Reduces manual effort in analyzing large performance datasets.